LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 6

DOUBLY LINKED LIST



Disusun oleh:

Rafi Imam Nasrullah

109082530010

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul X : https://github.com/RafiImamNasrullah/LAPRAK_RAFI_STRUKTUR-DATA/tree/9f713807f4f38d1f43711b5cf058731381af5d2f/MODUL%206

1. **Soal 1**

   Source Code

Main.cpp

```
#include <iostream>
#include "Doublylist.h"

using namespace std;

int main() {
   List L;
   infotype x;
   address P, Prec, Hapus;

   CreateList(L);



   for (int i = 1; i <= 4; i++) {
      cout << "\nMasukkan nomor polisi: ";
      cin >> x.nopol;

      cout << "Masukkan warna kendaraan: ";
      cin >> x.warna;

      cout << "Masukkan tahun kendaraan: ";
      cin >> x.thnBuat;

      if (findElm(L, x) != nullptr) {
         cout << "Nomor polisi sudah terdaftar" << endl;
      } else {
         P = alokasi(x);
         insertLast(L, P);
      }
   }

   printInfo(L);

   cout << "\nMasukkan Nomor Polisi yang dicari : ";
   cin >> x.nopol;

   P = findElm(L, x);
```

```
    if (P != nullptr) {
        cout << "\nNomor Polisi : " << P->info.nopol << endl;
        cout << "Warna       : " << P->info.warna << endl;
        cout << "Tahun       : " << P->info.thnBuat << endl;
    } else {
        cout << "Data kendaraan tidak ditemukan" << endl;
    }

    cout << "\nMasukkan Nomor Polisi yang akan dihapus : ";
    cin >> x.nopol;

    P = findElm(L, x);

    if (P != nullptr) {
        if (P == L.First) {
            deleteFirst(L, Hapus);
        } else if (P == L.Last) {
            deleteLast(L, Hapus);
        } else {
            Prec = P->prev;
            deleteAfter(Prec, Hapus);
        }

        dealokasi(Hapus);
        cout << "Data dengan nomor polisi " << x.nopol
             << " berhasil dihapus." << endl;
    } else {
        cout << "Data kendaraan tidak ditemukan." << endl;
    }

    printInfo(L);

    return 0;
}
```

Doublylist.h

```
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <iostream>
#include <string>
using namespace std;

struct infotype {
   string nopol;
   string warna;
   int thnBuat;
};

typedef struct ElmList *address;

struct ElmList {
   infotype info;
   address next;
   address prev;
};

struct List {
   address First;
   address Last;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);

address findElm(List L, infotype x);

void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);

#endif
```

Doublylist.cpp

```
#include "Doublylist.h"

void CreateList(List &L) {
   L.First = nullptr;
   L.Last = nullptr;
}

address alokasi(infotype x) {
   address P = new ElmList;

   P->info = x;
   P->next = nullptr;
   P->prev = nullptr;

   return P;
}

void dealokasi(address &P) {
   if (P != nullptr) {
      delete P;
      P = nullptr;
   }
}

void insertLast(List &L, address P) {
   if (P != nullptr) {
      if (L.First == nullptr) {
         L.First = P;
         L.Last = P;
      } else {
         P->prev = L.Last;
         L.Last->next = P;
         L.Last = P;
      }
   }
}

void printInfo(List L) {
   address P = L.Last;
```

```
   cout << "\nDATA LIST 1" << endl;

   while (P != nullptr) {
      cout << "\nNomor Polisi : " << P->info.nopol << endl;
      cout << "Warna       : " << P->info.warna << endl;
      cout << "Tahun       : " << P->info.thnBuat << endl;

      P = P->prev;
   }
}

address findElm(List L, infotype x) {
   address P = L.First;

   while (P != nullptr) {
      if (P->info.nopol == x.nopol) {
         return P;
      }

      P = P->next;
   }

   return nullptr;
}

void deleteFirst(List &L, address &P) {
   if (L.First != nullptr) {
      P = L.First;

      if (L.First == L.Last) {
         L.First = nullptr;
         L.Last = nullptr;
      } else {
         L.First = L.First->next;
         L.First->prev = nullptr;
      }

      P->next = nullptr;
      P->prev = nullptr;
   } else {
      P = nullptr;
   }
}
```

```
void deleteLast(List &L, address &P) {
   if (L.Last != nullptr) {
      P = L.Last;

      if (L.First == L.Last) {
         L.First = nullptr;
         L.Last = nullptr;
      } else {
         L.Last = L.Last->prev;
         L.Last->next = nullptr;
      }

      P->next = nullptr;
      P->prev = nullptr;
   } else {
      P = nullptr;
   }
}

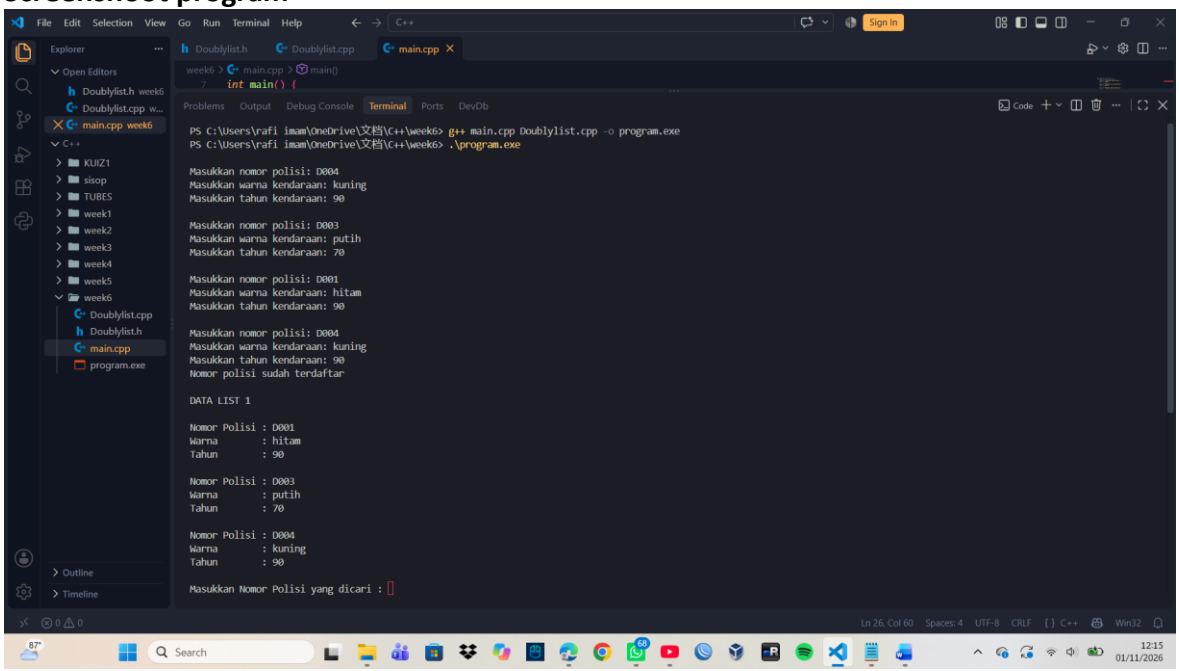
void deleteAfter(address Prec, address &P) {
   if (Prec != nullptr && Prec->next != nullptr) {
      P = Prec->next;

      Prec->next = P->next;

      if (P->next != nullptr) {
         P->next->prev = Prec;
      }

      P->next = nullptr;
      P->prev = nullptr;
   } else {
      P = nullptr;
   }
}
```

Screenshoot program



Deskripsi program

Program ini digunakan untuk mencari data kendaraan berdasarkan nomor polisi. Proses pencarian dilakukan dengan menelusuri elemen dari awal list menggunakan pointer next. Jika nomor polisi yang dicari ditemukan, program akan menampilkan nomor polisi, warna, dan tahun kendaraan. Jika data tidak ditemukan, program akan menampilkan pesan bahwa kendaraan tidak ditemukan.

2. Soal 2

Source Code

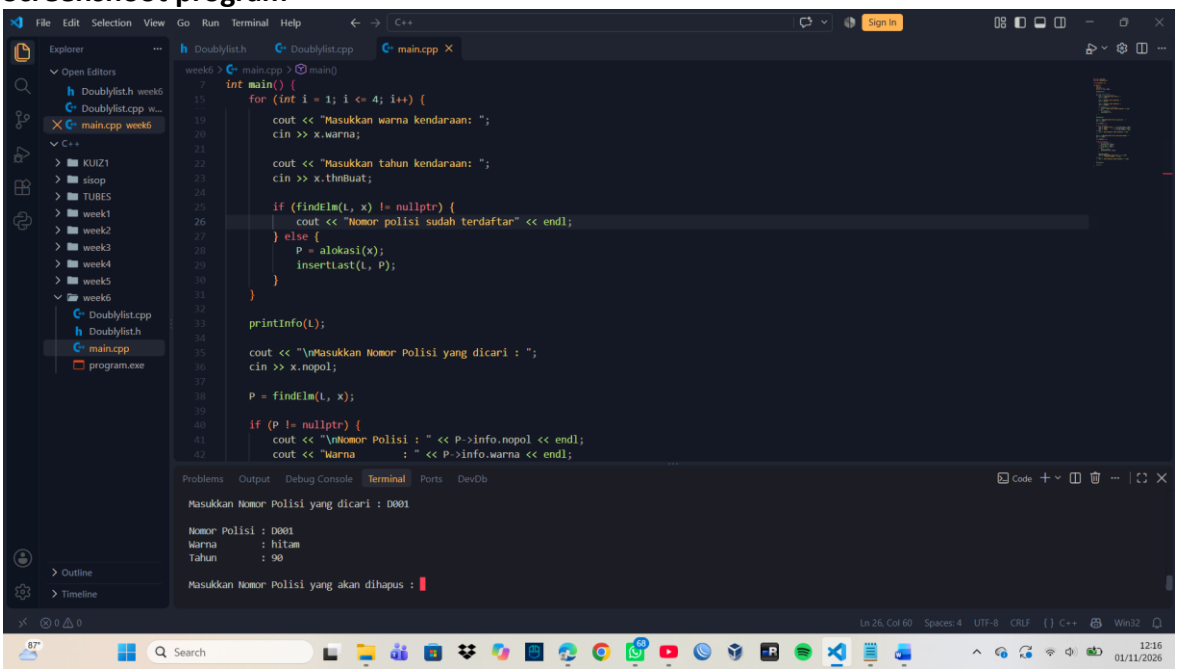
```
address findElm(List L, infotype x) {
   address P = L.First;

   while (P != nullptr) {
      if (P->info.nopol == x.nopol) {
         return P;
      }

      P = P->next;
   }

   return nullptr;
}
```

Screenshoot program



Deskripsi program

Program ini digunakan untuk mencari data kendaraan berdasarkan nomor polisi. Proses pencarian dilakukan dengan menelusuri elemen dari awal list menggunakan pointer next. Jika nomor polisi yang dicari ditemukan, program akan menampilkan nomor polisi, warna, dan tahun kendaraan. Jika data tidak ditemukan, program akan menampilkan pesan bahwa kendaraan tidak ditemukan.

3. **Soal 3**

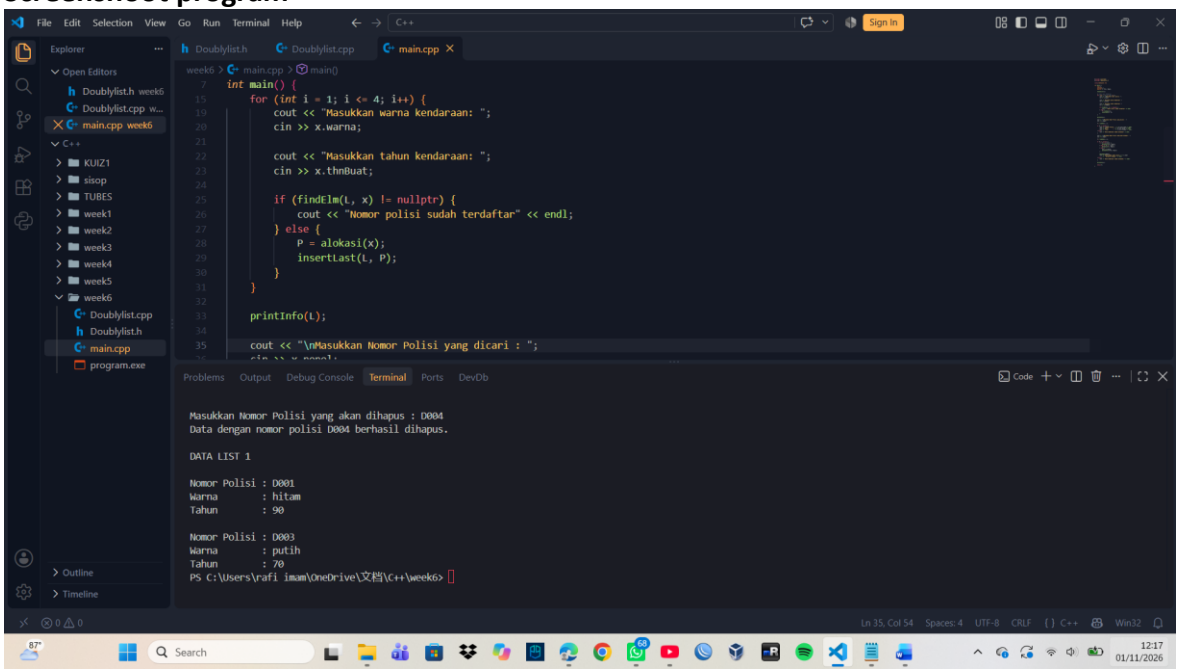
Source Code

```
void deleteFirst(List &L, address &P) {
   if (L.First != nullptr) {
      P = L.First;

      if (L.First == L.Last) {
         L.First = nullptr;
         L.Last = nullptr;
      } else {
         L.First = L.First->next;
         L.First->prev = nullptr;
      }

      P->next = nullptr;
      P->prev = nullptr;
   } else {
      P = nullptr;
   }
}
```

Screenshoot program



Deskripsi program

Program ini digunakan untuk menghapus data kendaraan dari Doubly Linked List. Terdapat tiga prosedur penghapusan, yaitu deleteFirst untuk menghapus elemen pertama, deleteLast untuk menghapus elemen terakhir, dan deleteAfter untuk menghapus elemen setelah posisi tertentu. Setelah elemen berhasil dihapus, fungsi dealokasi digunakan untuk membebaskan memori yang sebelumnya digunakan oleh elemen tersebut.